Script-Controlled ACL – Restrict Record Access Based on Field Value

ServiceNow Lab Documentation

1. Project Overview

This ServiceNow lab demonstrates script-controlled Access Control Lists (ACLs) to restrict access to records based on user roles and the value of a specific field. Users with the **bb1** role are permitted to access EEE records, while administrators retain full access. Additional roles control create, write, and delete operations.

2. Objectives

• Create users and custom roles.

• Create a custom institution/student table.

• Create READ, CREATE, WRITE, and DELETE ACLs.

• Use scripts to control record-level access.

• Restrict READ access based on the Branch field.

• Verify ACL behavior using user impersonation.

3. Technologies Used

ServiceNow • Access Control Lists (ACL) • JavaScript • Roles • Custom Tables • User Administration

4. User Creation

Navigate to **User Administration** → **Users** → **New**.

Field	Value
User ID	EEE User
First Name	EEE
Last Name	User
Email	eeeuser@gmail.com

5. Role Creation

Create roles **bb1**, **bb2**, **bb3**, and **bb4** under User Administration → Roles.

Role	Purpose
bb1	READ access
bb2	CREATE access
bb3	WRITE access
bb4	DELETE access

6. Custom Table Creation

Navigate to **System Definition** → **Tables** → **New**.

Field	Value
Label	Institution Details

Field	Value
Name	u_institution_details
Extends Table	False

7. Table Fields

Field	Type	Description
Student Roll Number	Auto Number	Automatically generated
Student Name	Reference	User
Faculty Name	Reference	User
Branch	Choice	ECE, EEE, CSE
Email	String	Student email
Phone Number	String	Student phone
Description	Multi String	Additional information

Create multiple records with EEE, ECE, and CSE branch values.

8. READ ACL

Field	Value
Type	Record
Operation	Read
Name	u_institution_details
Active	True
Advanced	True
Required Role	bb1
Data Condition	Branch is EEE

ACL Script:

```
(function () {
    // Allow admin users full access
    if (gs.hasRole('admin')) {
        return true;
    }

    // Allow users with bb1 role
    if (gs.hasRole('bb1')) {
        return true;
    }

    // Deny access for all others
    return false;
})();
```

READ ACL Verification

Impersonate the EEE User and open **u_institution_details.list**. Then impersonate an Admin user.

User	Expected Access
User with bb1	Can view EEE records
User without bb1	Cannot access records
Admin	Can view all records

9. CREATE ACL

Field	Value
Type	Record
Operation	Create
Name	u_institution_details
Active	True
Required Role	bb2
Data Condition	None

Verification: A user with bb1 + bb2 can view EEE records and see the New button.

10. WRITE ACL

Field	Value
Type	Record
Operation	Write

Field	Value
Name	u_institution_details
Active	True
Required Role	bb3
Data Condition	None

Verification: A user with bb1 + bb2 + bb3 can view and edit accessible EEE records.

11. DELETE ACL

Field	Value
Type	Record
Operation	Delete
Name	u_institution_details
Active	True
Required Role	bb4
Data Condition	None

Verification: A user with bb1 + bb2 + bb3 + bb4 can view, create, edit, and delete accessible records.

12. Final Permission Model

Operation	Required Role	Access
READ	bb1	View EEE records
CREATE	bb2	Create records
WRITE	bb3	Edit records
DELETE	bb4	Delete records
ADMIN	admin	Full access

13. Testing Checklist

■ READ: bb1 user can access intended EEE records.

■ CREATE: bb2 user sees the New button and can create records.

■ WRITE: bb3 user can edit accessible records.

■ DELETE: bb4 user can delete accessible records.

■ ADMIN: administrator can access all records and operations.

14. Learning Outcomes

• Understand ServiceNow ACLs.

• Create and assign custom roles.

• Create custom tables and fields.

• Control READ, CREATE, WRITE, and DELETE operations.

• Use JavaScript inside ACL scripts.

• Combine roles and record conditions for security.

• Test ACL behavior using user impersonation.

15. Outcome

This lab demonstrates how script-controlled READ, CREATE, WRITE, and DELETE ACLs can enforce record-level security in ServiceNow. The Institution Details table is protected according to user roles and branch information, while administrators retain full access.

16. Suggested GitHub Repository Structure

```
Script-Controlled-ACL/
■■■ README.md
■■■ ACL/
■   ■■■ Read-ACL.txt
■   ■■■ Create-ACL.txt
■   ■■■ Write-ACL.txt
■   ■■■ Delete-ACL.txt
■■■ Screenshots/
■   ■■■ User-Creation.png
■   ■■■ Role-Creation.png
■   ■■■ Table-Creation.png
■   ■■■ Read-ACL.png
■   ■■■ Create-ACL.png
■   ■■■ Write-ACL.png
■   ■■■ Delete-ACL.png
■■■ Documentation/
    ■■■ Test-Results.md
```